

Project Report: AIML Weather Assistant

Course: Fundamentals of AI and ML **Assignment:** Bring Your Own Project (BYOP) **Student**

Name: Amaan raza khan **Registration Number:** 25BAI11152 **GitHub Repository:**

<https://github.com/amaan25bai11152-art/AIML-Weather-Assistant>

1. Introduction and Problem Statement

The Problem: Unpredictable weather affects daily decision-making, from agricultural planning to simple daily commutes. While there are massive, highly complex meteorological models available, there is a distinct need for lightweight, transparent, and easily deployable weather prediction systems that can operate locally without heavy computational requirements.

Why it Matters: Many modern AI models act as "black boxes," making it difficult to understand *why* a specific prediction was made. For critical local systems, having a transparent decision-making process is highly valuable. This project addresses the need for a lightweight, transparent weather assistant by applying core Artificial Intelligence concepts to create a locally hosted prediction and tracking tool.

The AIML Weather Assistant is a Python-based AI application designed to predict weather conditions based on real-time user inputs of temperature and humidity. Furthermore, the system features a dynamic data logging and visualization pipeline, allowing it to record historical data, calculate rolling averages, and generate trend graphs.

2. Objectives

- To identify a real-world problem (weather prediction/tracking) and solve it using concepts learned in the Fundamentals of AI and ML course.
- To develop a functioning Rule-Based Expert System using Python.
- To implement persistent data storage using CSV file handling.
- To create dynamic, real-time data visualizations using the `matplotlib` library.
- To apply modular programming principles by dividing the application into distinct, purposeful scripts.

3. Methodology & Key Decisions

A key design decision in this project was the use of a **Rule-Based Expert System** rather than a statistical machine learning model (like Linear Regression or a Random Forest).

In statistical ML, a model must be trained on thousands of rows of data to "learn" mathematical correlations, which are then exported as a `.pkl` file. However, basic weather classification often follows strict meteorological thresholds.

Key Decision: I decided to build a decision tree utilizing strict conditional statements. By using a rule-based approach, the AI's decision-making process is 100% transparent, requires zero training time, and guarantees predictable, accurate outputs based on programmed expertise. Therefore, the custom logic script perfectly replaces the need for a `.pkl` file in this architecture, keeping the application lightweight and highly efficient.

4. System Architecture and Modules

To ensure code organization and documentation (a key BYOP evaluation criteria), the project is structured using a modular approach, separating the user interface, the AI logic, and the data visualization into distinct files.

- `app.py` (**Main Controller**): This is the central execution script. It serves as the bridge between the user, the AI logic, and the dataset. Its primary functions include reading historical data to display averages, utilizing `try/except` blocks to safely validate user inputs, and orchestrating the flow between the logic and visualization modules.
- `model_logic.py` (**The AI Engine**): This module acts as the core "brain" of the expert system. It evaluates the temperature (T) and humidity (H) to classify the weather into distinct categories (e.g., Snow, Cold, Rain, Sunny, Cloudy, or Normal).
- `analysis.py` (**Visualization Module**): This script utilizes the `matplotlib.pyplot` library to provide a visual representation of the stored data. It reads the updated CSV file and plots line graphs showing temperature and humidity trends over time.
- `dataset/history.csv` (**Data Storage**): A local CSV file that acts as the system's memory. Every time the program runs, the new data is appended as a new row, allowing the dataset to grow organically with user interaction.

5. Challenges Faced

During the development process, I encountered and overcame several technical challenges:

1. **Input Validation and Error Handling:** Initially, if a user entered a string instead of a numerical value for temperature, the entire program would crash. I overcame this by implementing robust `try/except` blocks in `app.py` to catch `ValueError` exceptions, ensuring the system prompts the user again instead of terminating unexpectedly.
2. **Data Persistence Integrity:** When writing to the CSV file, I initially faced issues with overwriting past data. I had to research file handling in Python to ensure I was opening the

`history.csv` file in append mode (`'a'`) rather than write mode (`'w'`), preserving the historical data pipeline.

3. **Visualization Integration:** Integrating `matplotlib` to run smoothly at the end of the terminal execution required careful handling. I had to ensure the graph pulled the most recently appended data and that closing the graph window properly terminated the script without hanging the terminal.

6. What I Learned

Building this capstone project provided several profound learning experiences:

- **AI Beyond Machine Learning:** I learned that Artificial Intelligence is not strictly limited to neural networks and statistical models. Rule-Based Expert Systems are a highly valid and effective form of AI when the problem domain has clear, definable logic.
- **End-to-End Data Pipelining:** I gained practical experience in building a complete data pipeline: taking user input → passing it through an AI logic model → securely storing it → and finally visualizing it for human analysis.
- **Modular Programming:** I learned the immense value of separating code into distinct files (`app.py` , `model_logic.py` , `analysis.py`). This made debugging significantly easier and my codebase much cleaner.

7. Output and Results

When the application is executed, the following sequence occurs successfully:

1. **Averages Displayed:** The system accurately reads past entries and prints historical averages to the terminal.
2. **Prediction Generated:** The AI engine processes new user inputs and outputs the accurate weather prediction based on the logic constraints.
3. **Data Persistence:** The system appends the new data to the CSV file seamlessly.
4. **Graphical Output:** A `matplotlib` window opens, plotting the historical timeline of both temperature and humidity, providing an immediate visual context.

8. Conclusion and Future Scope

The AIML Weather Assistant successfully demonstrates how logical AI, data persistence, and data visualization can be integrated into a cohesive, modular application to solve a real-world problem. The system is lightweight, highly accurate within its defined rules, and transparent in its operations.

Future enhancements could include:

- Expanding the rule base in `model_logic.py` to include wind speed and atmospheric pressure for more nuanced predictions.
- Transitioning from a terminal-based interface to a Graphical User Interface (GUI) using `tkinter` or `PyQt5`.